Exercise 4

SQL Fundamentals: Joins, UNION, Filtering, and Aggregates

Questions

1. SQL Joins

1. INNER JOIN

```
SELECT
        A.EmployeeID,
        A.FirstName,
        A.LastName,
        A.Department,
        A.Salary,
        B.ProjectID,
        B.ProjectName,
        B.Budget,
        B.Status
FROM    Employees AS A
INNER JOIN Projects AS B
ON A.EmployeeID = B.EmployeeID;
```

EmployeeID	FirstName	LastName	Department	Salary	ProjectID	ProjectName	Budget	Status
1	John	Doe	IT	70000	101	AI Development	100000	Completed
1	John	Doe	IT	70000	103	Cybersecurity Audit	75000	Pending
2	Alice	Smith	HR	60000	102	Employee Training	50000	Ongoing
3	Bob	Johnson	Finance	75000	104	Financial Analysis	90000	Ongoing
5	Emma	Wilson	Sales	65000	105	Market Expansion	65000	Completed
6	Michael	Clark	Finance	80000	106	Risk Management	80000	Pending

2. LEFT JOIN

```
SELECT
    A.EmployeeID,
    A.FirstName,
    A.LastName,
    A.Department,
    A.Salary,
    B.ProjectID,
    B.ProjectName,
    B.Budget,
    B.Status
FROM Employees AS A
LEFT JOIN Projects AS B
ON A.EmployeeID = B.EmployeeID;
```

EmployeeID	FirstName	LastName	Department	Salary	ProjectID	ProjectName	Budget	Status
1	John	Doe	IT	70000	101	AI Development	100000	Completed
1	John	Doe	IT	70000	103	Cybersecurity Audit	75000	Pending
2	Alice	Smith	HR	60000	102	Employee Training	50000	Ongoing
3	Bob	Johnson	Finance	75000	104	Financial Analysis	90000	Ongoing
4	David	Brown	IT	72000	NULL	NULL	NULL	NULL
5	Emma	Wilson	Sales	65000	105	Market Expansion	65000	Completed
6	Michael	Clark	Finance	80000	106	Risk Management	80000	Pending

3. RIGHT JOIN

```
SELECT
        B.ProjectID,
        B.ProjectName,
        B.Budget,
        B.Status,
        A.EmployeeID,
        A.FirstName,
        A.LastName,
        A.Department,
        A.Salary
FROM Employees AS A
RIGHT JOIN Projects AS B
ON A.EmployeeID = B.EmployeeID;
```

ProjectID	Project Name	Budget	Status	EmployeeID	FirstName	LastName	Department	Salary
101	AI Development	100000	Completed	1	John	Doe	IT	70000
102	Employee Training	50000	Ongoing	2	Alice	Smith	HR	60000
103	Cybersecurity Audit	75000	Pending	1	John	Doe	IT	70000
104	Financial Analysis	90000	Ongoing	3	Bob	Johnson	Finance	75000
105	Market Expansion	65000	Completed	5	Emma	Wilson	Sales	65000
106	Risk Management	80000	Pending	6	Michael	Clark	Finance	80000

4.FULL OUTER JOIN

```
SELECT
        A.EmployeeID,
        A.FirstName,
        A.LastName,
        A.Department,
        A.Salary,
        B.ProjectID,
        B.ProjectName,
        B.Budget,
        B.Status
FROM  Employees AS A
FULL OUTER JOIN Projects AS B
ON A.EmployeeID = B.EmployeeID ;
```

EmployeeID	FirstName	LastName	Department	Salary	ProjectID	ProjectName	Budget	Status
1	John	Doe	IT	70000	101	AI Development	100000	Completed
1	John	Doe	IT	70000	103	Cybersecurity Audit	75000	Pending
2	Alice	Smith	HR	60000	102	Employee Training	50000	Ongoing
3	Bob	Johnson	Finance	75000	104	Financial Analysis	90000	Ongoing
4	David	Brown	IT	72000	NULL	NULL	NULL	NULL
5	Emma	Wilson	Sales	65000	105	Market Expansion	65000	Completed
6	Michael	Clark	Finance	8000	106	Risk Management	80000	Pending

2. UNION & UNION ALL

5. UNION

```
SELECT
        City AS Location
FROM  Employees
UNION
SELECT
        Status AS Location
FROM  Projects;
```

Location
New York
Los Angeles
Toronto
London
Sydney
Completed
Ongoing
Pending

6. UNION ALL

```
SELECT
        City AS Location
FROM Employees
UNION ALL
SELECT
        Status AS Location
FROM Projects;
```

Location
New York
Los Angeles
Toronto
London
Sydney
New York
Completed
Ongoing
Pending
ongoing
Completed
Pending

3. Filtering Statements

```
7. SELECT
        EmployeeID,
        FirstName,
        LastName,
        Department,
        Salary
FROM Employees
WHERE Salary > 70000;
```

EmployeeID	FirstName	LastName	Department	Salary
3	Bob	Johnson	Finance	75000
4	David	Brown	IT	72000
6	Michael	Clark	Finance	80000

```
8. SELECT
        EmployeeID,
        FirstName,
        LastName,
        Department,
        Salary
FROM   Employees
WHERE  Department IN ('IT','Finance');
```

EmployeeID	FirstName	LastName	Department	Salary
1	John	Doe	IT	70000
3	Bob	Johnson	Finance	75000
4	David	Brown	IT	72000
6	Michael	Clark	Finance	80000

```
9. SELECT
        ProjectID,
        ProjectName,
        Budget,
        Status
FROM  Projects
WHERE Status <> 'Completed' ;
```

ProjectID	ProjectName	Budget	Status
102	Employee Training	50000	Ongoing
103	Cybersecurity Audit	75000	Pending
104	Financial Analysis	90000	Ongoing
106	Risk Management	80000	Pending

```
10. SELECT
        ProjectID,
        ProjectName,
        Budget,
        Status
FROM Projects
WHERE Budget > 70000
AND Status <> 'Completed' ;
```

ProjectID	ProjectName	Budget	Status
103	Cybersecurity Audit	75000	Pending
104	Financial Analysis	90000	Ongoing
106	Risk Management	80000	Pending

11. SELECT

```
    EmployeeID,
    FirstName,
    LastName,
    Department,
    Salary,
    City
FROM Employees
WHERE City IN ('New York', 'Toronto')
ORDER BY Salary DESC;
```

EmployeeID	FirstName	LastName	Department	Salary	City
6	Michael	Clark	Finance	80000	New York
3	Bob	Johnson	Finance	75000	Toronto
1	John	Doe	IT	70000	New York

13. SELECT

```
    EmployeeID,
    FirstName,
    LastName,
    Department,
    Salary
FROM Employees
ORDER BY Salary DESC
LIMIT 3;
```

EmployeeID	FirstName	LastName	Department	Salary
6	Michael	Clark	Finance	80000
3	Bob	Johnson	Finance	75000
4	David	Brown	IT	72000

4. Aggregate Functions with GROUP BY & HAVING

13. SELECT

```
        Department,
        SUM(Salary) AS TotalSalary
FROM Employees
GROUP BY Department
ORDER BY TotalSalary DESC;
```

Department	TotalSalary
Finance	155000
IT	142000
Sales	65000
HR	60000

14. SELECT

```
        City,
        AVG(Salary) AS AverageSalary,
FROM Employees
GROUP BY City
HAVING AVG(Salary) > 65000 ;
```

City	AverageSalary
New York	75000
Toronto	75000
London	72000

15. SELECT

Department,

COUNT(EmployeeID) AS EmployeeCount

FROM Employees

GROUP BY Department

HAVING COUNT(EmployeeID) > 1;

Department	EmployeeCount
IT	2
Finance	2

16. SELECT

Status,

COUNT(ProjectID) AS ProjectCount

FROM Projects

GROUP BY Status

HAVING COUNT(ProjectID) >= 2;

Status	ProjectCount
Completed	2
Ongoing	2
Pending	2

17. SELECT

```
        A.EmployeeID,
        A.FirstName,
        A.LastName,
        Sum(B.Budget) AS TotalProjectBudget
FROM    Employees AS A
INNER JOIN Projects AS B
ON A.EmployeeID = B.EmployeeID
GROUP BY A.EmployeeID, A.FirstName, A.LastName
HAVING Sum(B.Budget) > 150000;
```

EmployeeID	FirstName	LastName	TotalProjectBudget
1	John	Doe	175000